main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}

void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
```

Output

```
10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4


=== Code Execution Successful ===
```

main.c

Share Run

```c
void insertAtPosition(struct Node** head, int data, int position) {
    struct Node* newNode = createNode(data);
    if (position == 0) {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position out of range\n");
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}

void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}

void deleteFromEnd(struct Node** head) {
    if (*head == NULL) return;
    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    struct Node* temp = *head;
```

Output

```
10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4


=== Code Execution Successful ===
```

main.c

Share Run

```c
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}

void deleteFromPosition(struct Node** head, int position) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    if (position == 0) {
        *head = temp->next;
        free(temp);
        return;
    }
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range\n");
        return;
    }
    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}

void traverse(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
```

Output

```
10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4


=== Code Execution Successful ===
```

main.c

Share Run

```c
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}

void deleteFromPosition(struct Node** head, int position) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    if (position == 0) {
        *head = temp->next;
        free(temp);
        return;
    }
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range\n");
        return;
    }
    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}

void traverse(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
```

Output

```
10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4


=== Code Execution Successful ===
```

main.c

Share Run

```c
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}

void deleteFromPosition(struct Node** head, int position) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    if (position == 0) {
        *head = temp->next;
        free(temp);
        return;
    }
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Position out of range\n");
        return;
    }
    struct Node* next = temp->next->next;
    free(temp->next);
    temp->next = next;
}

void traverse(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
```

Output

```
10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4


=== Code Execution Successful ===
```

main.c

Share Run

```c
}

int search(struct Node* head, int target) {
    struct Node* temp = head;
    while (temp != NULL) {
        if (temp->data == target) return 1;
        temp = temp->next;
    }
    return 0;
}

void reverse(struct Node** head) {
    struct Node* prev = NULL;
    struct Node* current = *head;
    struct Node* next = NULL;
    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head = prev;
}

void sort(struct Node** head) {
    if (*head == NULL || (*head)->next == NULL) return;
    struct Node* temp = *head;
    while (temp != NULL) {
        struct Node* next = temp->next;
        while (next != NULL) {
            if (temp->data > next->data) {
                int data = temp->data;
                temp->data = next->data;
                next->data = data;
```

Output

```
10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4


=== Code Execution Successful ===
```

main.c

Share Run

```c
}

int search(struct Node* head, int target) {
    struct Node* temp = head;
    while (temp != NULL) {
        if (temp->data == target) return 1;
        temp = temp->next;
    }
    return 0;
}

void reverse(struct Node** head) {
    struct Node* prev = NULL;
    struct Node* current = *head;
    struct Node* next = NULL;
    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    *head = prev;
}

void sort(struct Node** head) {
    if (*head == NULL || (*head)->next == NULL) return;
    struct Node* temp = *head;
    while (temp != NULL) {
        struct Node* next = temp->next;
        while (next != NULL) {
            if (temp->data > next->data) {
                int data = temp->data;
                temp->data = next->data;
                next->data = data;
```

Output

```
10 -> 20 -> 30 -> NULL
5 -> 10 -> 20 -> 30 -> NULL
5 -> 10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> 30 -> NULL
10 -> 25 -> 20 -> NULL
10 -> 20 -> NULL
List contains 20: Yes
List contains 40: No
20 -> 10 -> NULL
20 -> 10 -> 15 -> 5 -> NULL
5 -> 10 -> 15 -> 20 -> NULL
Length of the list: 4


=== Code Execution Successful ===
```